

SENSOR BASED SIMULATION USING TINKERCAD/PROTEUS

Temperature sensor using LCD

Aim

To simulate an IoT-based Temperature sensor using LCD in Tinkercad/Proteus where it sense the room temperature

Objective

1. To understand sensor-based automation.
2. To learn Arduino interfacing with sensors.
3. To simulate IoT systems using Tinkercad/Proteus.
4. To sense the room temperature using LCD

Components Required

<u>S.No</u>	Component	Quantity
1.	Arduino Uno	1
2.	Breadboard	1
3.	Temperature Sensor	1
4.	Resistor (330 ohm)	1
5.	Potentiometer	1
6.	Jumper wires	Few

Software Required

- Tinkercad
- Proteus Design Suite
- Arduino IDE

Working Principle

1. The temperature sensor measures the surrounding temperature. Arduino reads the sensor output through A0.
2. The analog value is converted into voltage. It is converted into temperature in Celsius.
3. Temperature is displayed on the LCD and Serial Monitor. The process repeats continuously.

Circuit Connections

Temperature Sensor (TMP36)

TMP36 Pin

Arduino Connection

Left Pin (+Vs)

5V

Middle Pin (Vout)

A0

Right Pin (GND)

GND

LCD 16×2 Connections

LCD Pin

Arduino Connection

VSS (1)

GND

VDD (2)

5V

V0 (3)

Middle pin of 10kΩ potentiometer

RS (4)

Pin 8

RW (5)

GND

E (6)

Pin 7

D4 (11)

Pin 6

D5 (12)

Pin 5

D6 (13)

Pin 4

D7 (14)

Pin 3

A (15)

5V through 220Ω resistor

K (16)

GND

Potentiometer Connections (10kΩ)

Potentiometer Pin

Connection

Left Pin

5V

Middle Pin

LCD V0 (Pin 3)

Right Pin

GND

The potentiometer is used to adjust the LCD contrast.

Circuit Diagram Overview

If you are using an LM35 sensor instead of TMP36

Connections remain the same:

LM35 VCC → 5V

LM35 OUT → A0

LM35 GND → GND

Arduino program

```
#include "LiquidCrystal.h"
```

```
LiquidCrystal lcd(8,7,6,5,4,3);
```

```
int sensorPin = 0;
```

```
void setup()
```

```
{
```

```
  Serial.begin(9600);
```

```
  lcd.begin(16,2);
```

```
}
```

```
void loop()
```

```
{
```

```
  int reading = analogRead(sensorPin);
```

```
  // measure the 5v with a meter for an accurate value
```

```
  //In particular if your Arduino is USB powered
```

```
  float voltage = reading * 4.68;
```

```

voltage /= 1024.0;
// now print out the temperature
float temperatureC = (voltage - 0.5) * 100; Serial.print(temperatureC);
Serial.println(" degrees C");
lcd.setCursor(0,0);
lcd.print("Temperature Value ");
lcd.setCursor(0,1);
lcd.print(" degrees C");
lcd.setCursor(11,1);
lcd.print(temperatureC);
delay(100);
}

```

Code Explanation

1. Include the LCD Library

C++

```
#include "LiquidCrystal.h"
```

This line includes the LiquidCrystal library, which allows the Arduino to communicate with and control the LCD display.

2. Create LCD Object

C++

```
LiquidCrystal lcd(8,7,6,5,4,3);
```

This statement specifies the Arduino pins connected to the LCD:

RS → Pin 8

E → Pin 7

D4 → Pin 6

D5 → Pin 5

D6 → Pin 4

D7 → Pin 3

3. Define Sensor Pin

C++

```
int sensorPin = 0;
```

The temperature sensor output is connected to analog pin A0 of the Arduino.

4. Setup Function

C++

```
void setup()
```

```
{
```

```
    Serial.begin(9600);
```

```
    lcd.begin(16,2);
```

```
}
```

```
Serial.begin(9600);
```

Initializes serial communication at 9600 baud rate.

Used to display temperature values on the Serial Monitor.

```
lcd.begin(16,2);
```

Initializes the LCD with 16 columns and 2 rows.

5. Read Analog Sensor Value

```
C++
```

```
int reading = analogRead(sensorPin);
```

Reads the analog voltage from the sensor.

Returns a value between 0 and 1023.

6. Convert ADC Reading to Voltage

```
C++
```

```
float voltage = reading * 4.68;
```

```
voltage /= 1024.0;
```

The analog value is converted into voltage using:

Where:

Reading = Analog value from the sensor

4.68V = Reference voltage

1024 = ADC resolution of Arduino

7. Calculate Temperature

```
C++
```

```
float temperatureC = (voltage - 0.5) * 100;
```

This formula converts the voltage into temperature in Celsius.

For a TMP36 sensor:

0.5V corresponds to 0°C

Every 10 mV increase represents 1°C

Formula:

8. Display Temperature on Serial Monitor

```
C++
```

```
Serial.print(temperatureC);
```

```
Serial.println(" degrees C");
```

These statements print:

Temperature value

Unit (degrees Celsius)

Example Output:

28.5 degrees C

9. Display Data on LCD

```
C++
```

```
lcd.setCursor(0,0);
```

```
lcd.print("Temperature Value ");
```

Displays the heading on the first row.

```
C++
```

```
lcd.setCursor(0,1);
```

```
lcd.print(" degrees C");
```

Displays the temperature unit on the second row.

C++

```
lcd.setCursor(11,1);
```

```
lcd.print(temperatureC);
```

Displays the temperature value starting from column 11 of the second row.

Example LCD Display:

Temperature Value

28.5 degrees C

10. Delay

C++

```
delay(100);
```

Pauses the program for 100 milliseconds before taking the next temperature reading.

Simulation Steps in Tinkercad

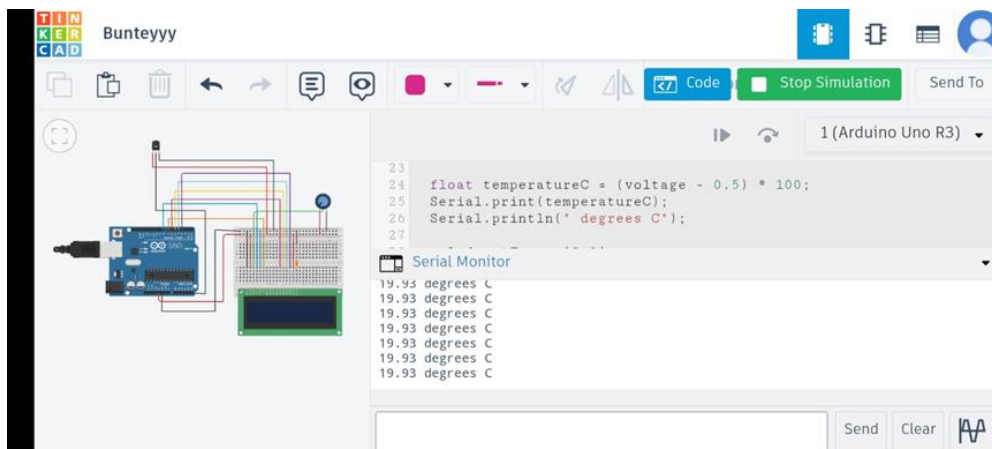
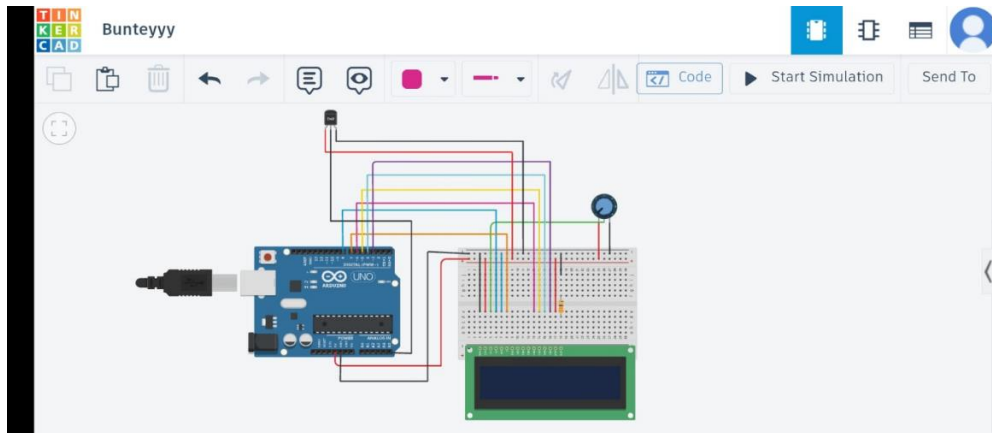
1. Open Tinkercad.
2. Click on "Create New Circuit".
3. Add Arduino Uno.
4. Add LCD, potentiometer and Resistors.
5. Make the circuit connections.
6. Copy and paste the Arduino code.
7. Click "Start Simulation".
8. Observe the temperature

Simulation Steps in Proteus

1. Open Proteus Design Suite.
2. Create a new project.
3. Add Arduino Uno.
4. Add LCD ,potentiometer etc.
5. Connect all components.
6. Compile Arduino code using Arduino IDE.
7. Generate HEX file.

8. Upload HEX file to Arduino in Proteus.
9. Run the simulation.
10. Observe the temperature.

Expected Output



Applications

- 1.Home temperature monitoring.
- 2.Weather monitoring stations.
- 3.Greenhouses for plant growth.
- 4.Industrial machine monitoring.
- 5.Cold storage temperature control.
- 6.Laboratory temperature measurement.
- 7.Smart home automation.
- 8.Educational and Arduino learning projects.

Advantages

Advantages of Temperature Monitoring System

1. Simple and easy to build.
2. Low-cost project.
3. Provides real-time temperature readings.
4. Easy to monitor temperature using an LCD display.
5. Accurate and reliable measurements.
6. Can be used in many applications such as homes, industries, and agriculture.
7. Low power consumption.
8. Easy to modify and upgrade with additional sensors or alarms.

Conclusion

The Temperature Monitoring System using Arduino is a simple, cost-effective, and reliable project for measuring and displaying temperature in real time. It helps in easy temperature monitoring and can be used in homes, industries, agriculture, and educational applications. This project demonstrates the practical use of sensors, Arduino, and LCD displays in real-world monitoring systems.